RAMDEOBABA UNIVERSITY, NAGPUR

Department of Electronics Engineering

CAPSTONE PROJECT PROPOSAL

Course: Deep Learning Lab | Academic Session: 2026-27 | Semester: V

Project Title:	Modelling Malware Concept Drift Using Neural ODEs
Team Member 1:	Name: Rinkle Jain Roll No.: 42
Team Member 2:	Name: Rohan Pendharkar Roll No.: 43
Team Member 3:	Name: Rakshit Waze Roll No.: 40
Team Member 4:	Name: Neha Kewalramani Roll No.: 32

1. Problem Statement and Motivation

Malware detection models trained with machine learning usually score above 99% accuracy when tested in the lab, but their performance drops sharply once deployed, sometimes by close to 50% within a few months. This happens because attackers keep changing their code, packers and delivery methods, so the relationship between a file's features and whether it is malicious keeps shifting. This is called concept drift. Most existing classifiers (Random Forest, XGBoost, MLPs) learn one fixed decision boundary and never update it, so they slowly become outdated. Even random-forest based classifiers cannot tell when or how the malware landscape is moving. This project tries to model that movement directly, using a continuous-time model (Neural ODE) that learns how malware features evolve over time, instead of freezing a boundary at one point in time.

2. Project Objectives

1. To design a Latent Neural ODE model that represents malware feature drift as a continuous process rather than a fixed decision boundary.

2. To train and test the model using strict month-wise (chronological) data splits so that future malware samples never leak into training, unlike random k-fold splits.

3. To measure how much the model's accuracy decays over time using the Area Under Time (AUT) metric, and compare this decay against static baseline models (XGBoost, LightGBM, MLP).

4. To handle the class imbalance in the dataset (malware vs benign files) using Focal Loss so rare malware samples are not ignored during training.

5. (Stretch goal, if time permits) To add a jump-update mechanism to the ODE so it can react to sudden, one-time drift events, such as the spike in malware activity seen around the COVID-19 period in the dataset.

3. Dataset

Dataset Name	BODMAS (Blue Hexagon Open Dataset for Malware Analysis)
Dataset Source	Publicly available, Yang et al. (2021), GitHub: whyisyoung/BODMAS
Dataset Size	134,435 samples (57,293 malware + 77,142 benign), collected over 14 months (Aug 2019 - Sep 2020)
Type of Data	Tabular / Time-Series - each file is a pre-extracted 2,381-dimensional numeric feature vector (from PE header, entropy, imports, byte histograms)
No. of Classes	2 (Benign, Malicious) - binary classification

We picked BODMAS over bigger datasets like EMBER or SOREL-20M because it has a proper timestamp for every sample, spread across 14 continuous months. That timeline is exactly what is needed for an ODE-based model, since the whole point is to learn how the feature space moves month to month, not just classify a random shuffled set of files. The dataset is public and does not need any special access, and since it's only structural file metadata, there's no privacy or PII concern.

4. Proposed Methodology

Raw PE Features (2381-d) → Encoder (compress to latent state) → Neural ODE Solver (evolve latent state through time) → Decoder → Benign / Malicious Prediction

Proposed Model / Architecture: Latent Neural ODE (encoder-solver-decoder pipeline).

An encoder network first compresses the 2,381 raw features of a file into a smaller latent vector (around 64 dimensions). Instead of feeding this into a normal classifier straight away, we treat this latent vector as the starting point of a differential equation and let a small neural network (the ODE function) describe how this state should keep changing as time moves forward. An ODE solver (Dormand-Prince / dopri5) then integrates this equation to project the file's latent representation forward to the time it needs to be classified at. A decoder (a simple MLP) then takes this evolved latent state and outputs whether the file is benign or malicious.

Brief justification: A normal classifier only knows about the training months it saw. An RNN/LSTM also assumes fixed and evenly spaced time steps, but malware doesn't appear on a fixed schedule; sometimes there are ten variants in one week and then nothing for two months. A Neural ODE does not have this constraint since it works in continuous time and can be evaluated at any gap. To make backpropagation possible without running out of GPU memory, we use the adjoint sensitivity method, which keeps memory usage constant no matter how many integration steps the solver takes. This is what makes it possible to train this on a single consumer GPU instead of needing a cluster.

5. Experimental and Evaluation Plan

Data Split	Training: Months 0-7 (~57%) Validation: Months 8-9 (~14%) Testing: Months 10-13 (~29%) - split chronologically, not randomly
Loss Function	Focal Loss (handles the imbalance between benign and malicious samples better than plain cross-entropy)
Optimizer	Adam, with the ODE trained through the adjoint sensitivity method
Evaluation Metrics	AUROC, Macro F1-score, and Area Under Time (AUT) to track performance decay month by month
Baseline / Comparison	XGBoost, LightGBM, and a parameter-matched MLP, all trained on the same chronological split

The main experiment is simple: train every model (ours and the baselines) only on the first eight months, then test all of them on the last four months, which the models have never seen. We expect the static models (XGBoost, MLP) to show a clear drop in AUROC as we move further into the test months, while we want to see whether the Neural ODE model's AUROC stays flatter across this period. We will also do a couple of ablations, like removing the ODE solver to check that it is actually the continuous integration helping and not just the encoder/decoder network.

6. Expected Outcome / Deliverables

- A trained and evaluated Latent Neural ODE model for malware classification, along with the static baseline models trained on the same data.

- Month-by-month AUROC / AUT plots comparing how much each model's performance drops over the four test months.

- A written comparison of the ODE model against the static baselines, including cases where the ODE model does not help much, and why.

- If time allows, a small extension where the model reacts to a sudden drift event (the COVID-19 period spike in the dataset) using a jump-update mechanism, along with a UMAP visualization of how the malware clusters move in the latent space over time.

7. Work Distribution

Sr. No.	Team Member	Major Responsibility
1	Member 1	Data collection, preprocessing, and building the chronological train/val/test splits
2	Member 2	Implementing and training the static baseline models (XGBoost, LightGBM, MLP)
3	Member 3	Implementing the Latent Neural ODE (encoder, solver, decoder) and the adjoint training loop
4	Member 4	Evaluation (AUT metric, plots), ablations, and final report/presentation

Every member will stay involved in all stages and be able to explain the complete project, not just their assigned part.

8. Proposed Timeline

Phase	Proposed Activity
Phase 1	Problem finalization, data acquisition, and building chronological splits
Phase 2	Preprocessing and static baseline implementation (XGBoost, LightGBM, MLP)
Phase 3	Neural ODE model development and training (encoder, solver, decoder, adjoint method)
Phase 4	Experimentation: AUT evaluation, baseline comparison, solver/ablation tuning
Phase 5	Error analysis and, if time permits, the jump-network extension
Phase 6	Final demonstration, report writing and presentation

9. References

[1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, "Neural Ordinary Differential Equations," NeurIPS, 2018.

[2] Y. Rubanova, R. T. Q. Chen, and D. Duvenaud, "Latent Ordinary Differential Equations for Irregularly-Sampled Time Series," NeurIPS, 2019.

[3] C. Herrera, F. Krach, and J. Teichmann, "Neural Jump Ordinary Differential Equations," arXiv:2006.04727, 2020.

[4] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," USENIX Security, 2019.

[5] L. Yang, A. Ciptadi, I. Laziuk, A. Ahmadzadeh, and G. Wang, "BODMAS: An Open Dataset for Learning-based Temporal Analysis of PE Malware," IEEE DLS, 2021.

[6] BODMAS Dataset repository, https://github.com/whyisyoung/BODMAS